ข้อสอบปฏิบัติ

วิชา การออกแบบและการพัฒนาเว็บไซต์

(Web Design and Development)

รายการ	รายละเอียด
คะแนนเต็ม	100 คะแนน
เวลาพัฒนา	3 ชั่วโมง
รูปแบบ	ปฏิบัติ — อนุญาตให้ใช้ AI ช่วยพัฒนาได้ ภายใต้เงื่อนไขในส่วนที่ 3
ชื่อ-สกุล / รหัส	นายณัฐพล คำรังศรี รหัส 680112230032
หัวข้อเว็บไซต์ที่เลือก	คลินิกสัตว์เลี้ยง

ส่วนที่ 1: โจทย์

สถานการณ์

ท่านได้รับมอบหมายให้พัฒนาเว็บไซต์ "ศูนย์รวมข้อมูลท่องเที่ยวชุมชนจังหวัดบุรีรัมย์" (หรือหัวข้อเทียบเท่าที่ผู้สอนอนุมัติ เช่น ร้านกาแฟ ร้านหนังสือ คลินิกสัตว์เลี้ยง)

เพื่อเผยแพร่ข้อมูลสถานที่ ข่าวสาร และข้อมูลสภาพอากาศแก่ผู้เข้าชม

ข้อกำหนดการพัฒนา

1. โครงสร้างเว็บไซต์ (อย่างน้อย 3 หน้า)

- หน้าแรก (Home) — index.html
- หน้ารายการ/แสดงข้อมูล (Listing) — ต้องมีการแสดงข้อมูลจาก API อย่างน้อย 1 จุดในเว็บไซต์ (หน้าใดก็ได้)
- หน้าติดต่อเรา (Contact) — มีแบบฟอร์มพร้อมตรวจสอบข้อมูล
- ทุกหน้ามีเมนูนำทางและส่วนท้าย (Footer) เหมือนกัน

2. HTML / CSS / JavaScript

- ใช้ HTML5 semantic tags (header, nav, main, section, article, footer)
- จัดวางด้วย Flexbox หรือ Grid
- JavaScript อย่างน้อย 3 ฟังก์ชัน (ไม่นับการเรียก API) เลือกจาก: ค้นหา/กรองข้อมูล, เมนูมือถือ (hamburger), ตรวจสอบแบบฟอร์ม (validation), สลับโหมดสว่าง/มืด

3. เฟรมเวิร์คและไลบรารี

- ใช้อย่างน้อย 1 ตัว เช่น Bootstrap, Tailwind CSS, Font Awesome, AOS, Swiper.js
- เขียนเหตุผลที่เลือกใช้ไว้ในไฟล์ README.md (2-3 ประโยค)

4. การออกแบบส่วนติดต่อผู้ใช้ (UI/UX)

- กำหนดชุดสีและฟอนต์ที่สอดคล้องกัน (ไม่เกิน 2 ฟอนต์)
- ปุ่มและลิงก์มีสถานะ hover/focus ชัดเจน
- มีการจัดลำดับความสำคัญของเนื้อหา (visual hierarchy) และคอนทราสต์ที่อ่านง่าย

5. Responsive Design

- รองรับอย่างน้อย 3 ขนาด: มือถือ (≤576px), แท็บเล็ต (≤992px), เดสก์ท็อป
- รูปภาพปรับขนาดตามหน้าจอ ไม่มีการเลื่อนแนวนอน
- ใช้ <meta name="viewport"> และ Media Query

6. การเชื่อมต่อ API

- ดึงข้อมูลจาก API สาธารณะอย่างน้อย 1 แหล่งด้วย fetch() หรือ axios — แนะนำ Open-Meteo (ไม่ต้องใช้ API key)
- แสดงข้อมูลบนหน้าเว็บ พร้อมสถานะ Loading และจัดการข้อผิดพลาด (error handling) เมื่อเรียกไม่สำเร็จ
- ใช้ async/await หรือ Promise

7. SEO เบื้องต้น

- <title> และ <meta name="description"> ไม่ซ้ำกันในแต่ละหน้า
- มี h1 เพียงหนึ่งเดียวต่อหน้า และ heading เรียงลำดับถูกต้อง
- รูปภาพทุกรูปมี alt; ชื่อไฟล์สื่อความหมาย; มี Open Graph meta อย่างน้อย 2 ตัว (เช่น og:title, og:description)

8. การทดสอบและแก้ไขข้อผิดพลาด

- ทดสอบด้วย Chrome DevTools (Device Toolbar, Console)
- ตรวจด้วย Lighthouse (หน้าแรก) และ W3C HTML Validator
- บันทึกตารางทดสอบอย่างน้อย 3 รายการ (ปัญหาที่พบ ⟶ วิธีแก้ไข) ใน README.md

ส่วนที่ 2: แนะนำการจัดสรรเวลา (180 นาที)

ช่วงเวลา	งานที่ควรเสร็จ
0:00 – 0:10	อ่านโจทย์ เลือกหัวข้อ ร่าง Sitemap และเลือกเฟรมเวิร์ค
0:10 – 0:55	สร้างหน้า Home และ Listing (HTML + CSS + Responsive เบื้องต้น)
0:55 – 1:30	หน้า Contact + JavaScript 3 ฟังก์ชัน ★ Checkpoint 1: โครงทุกหน้าเสร็จ
1:30 – 2:05	เชื่อมต่อ API + Loading/Error ★ Checkpoint 2: JS และ API ทำงานได้
2:05 – 2:25	SEO ทุกหน้า
2:25 – 2:45	ทดสอบ DevTools / Lighthouse / W3C และแก้ไขข้อผิดพลาด
2:45 – 3:00	เขียน README.md และบันทึกการใช้ AI ★ Checkpoint 3: ส่งงาน

ส่วนที่ 3: ข้อกำหนดการใช้ AI

อนุญาตให้ใช้ AI (ChatGPT, Claude, Copilot, Gemini ฯลฯ) ช่วยพัฒนาได้ โดยต้องปฏิบัติดังนี้

- **บันทึกการใช้ AI อย่างน้อย 3 รายการ** ระบุ Prompt ที่ใช้ ผลลัพธ์ที่ได้ และสิ่งที่นักศึกษาแก้ไข/ปรับปรุงเอง (ไฟล์ AI_LOG.md)
- **อธิบายโค้ดได้ (Viva)** ผู้สอนจะสุ่มชี้โค้ดให้อธิบาย 3-5 นาที หากอธิบายไม่ได้ จะถูกหักคะแนนส่วนที่เกี่ยวข้อง
- **ห้ามคัดลอกผลงานผู้อื่น** หรือใช้เทมเพลตสำเร็จรูปทั้งชุดโดยไม่ดัดแปลง

ส่วนที่ 4: สิ่งที่ต้องส่ง

รายการ	รูปแบบ
ซอร์สโค้ดเว็บไซต์	โฟลเดอร์ .zip หรือ GitHub repository
README.md	Sitemap, เหตุผลเลือกเฟรมเวิร์ค, ตารางทดสอบ ≥3 รายการ
AI_LOG.md	บันทึกการใช้ AI ≥3 รายการ
ภาพหน้าจอ Lighthouse	1 ภาพ (หน้าแรก) วางในโฟลเดอร์ docs/ หรือแนบใน README
ไฟล์นี้	ใส่ชื่อและรายละเอียดของงาน
ลิงก์เว็บไซต์ที่เผยแพร่	ไม่บังคับ (GitHub Pages / Netlify / Vercel)

Zip รวมกันส่งใน lms2.bru.ac.th

ส่วนที่ 5: เกณฑ์การตรวจให้คะแนน (Rubric) รวม 100 คะแนน

5.1 โครงสร้างเว็บไซต์และ HTML (10 คะแนน)

เกณฑ์ (คะแนน)	ดีมาก	พอใช้	ต้องปรับปรุง
จำนวนหน้าและการเชื่อมโยง (4)	ครบ ≥3 หน้า ลิงก์ถูกต้องทุกหน้า (4)	ครบแต่ลิงก์เสียบางจุด (2-3)	ไม่ครบ/ลิงก์เสียมาก (0-1)
Semantic HTML (4)	ใช้แท็กถูกความหมายทั้งเว็บ (4)	ใช้บางส่วน (2-3)	ใช้ div เกือบทั้งหมด (0-1)
โครงสร้างโฟลเดอร์/ชื่อไฟล์ (2)	เป็นระเบียบ (2)	พอใช้ (1)	สับสน (0)

5.2 CSS และการออกแบบ UI (15 คะแนน)

เกณฑ์ (คะแนน)	ดีมาก	พอใช้	ต้องปรับปรุง
การจัดวางด้วย Flexbox/Grid (5)	ใช้ถูกต้อง จัดวางสวยงาม (5)	ใช้ได้แต่ไม่สม่ำเสมอ (3-4)	จัดวางผิดพลาด (0-2)
ชุดสี ฟอนต์ ความสอดคล้อง (5)	สอดคล้องทั้งเว็บ อ่านง่าย คอนทราสต์ดี (5)	พอใช้ (3-4)	ไม่สอดคล้อง (0-2)
Visual Hierarchy และ UX (5)	หาข้อมูลง่าย มี hover/focus ชัดเจน (5)	พอใช้ (3-4)	สับสน (0-2)

5.3 JavaScript และการโต้ตอบ (15 คะแนน)

เกณฑ์ (คะแนน)	ดีมาก	พอใช้	ต้องปรับปรุง
จำนวนและความถูกต้องของฟังก์ชัน (8)	≥3 ฟังก์ชัน ทำงานถูกต้อง ไม่มี error (8)	2 ฟังก์ชัน หรือมีข้อบกพร่อง (4-7)	≤1 ฟังก์ชัน/ใช้งานไม่ได้ (0-3)
คุณภาพโค้ด (4)	อ่านง่าย แยกฟังก์ชัน มี comment (4)	พอใช้ (2-3)	สับสน (0-1)
Form Validation (3)	ตรวจครบ แจ้งเตือนชัดเจน (3)	ตรวจบางช่อง (1-2)	ไม่มี (0)

5.4 เฟรมเวิร์คและไลบรารี (8 คะแนน)

เกณฑ์ (คะแนน)	ดีมาก	พอใช้	ต้องปรับปรุง
ใช้ถูกต้องและเหมาะสม (5)	ใช้ถูกวิธี เสริมคุณภาพงานจริง (5)	ใช้แต่ไม่เต็มประสิทธิภาพ (3-4)	ใส่ลิงก์แต่ไม่ได้ใช้งาน (0-2)

เกณฑ์ (คะแนน)	ดีมาก	พอใช้	ต้องปรับปรุง
เหตุผลใน README (3)	ชัดเจนมีเหตุผล (3)	อธิบายสั้น (1-2)	ไม่อธิบาย (0)

5.5 Responsive Design (12 คะแนน)

เกณฑ์ (คะแนน)	ดีมาก	พอใช้	ต้องปรับปรุง
รองรับ 3 ขนาดหน้าจอ (6)	แสดงผลดีทั้ง 3 ขนาด ไม่เลื่อนแนวนอน (6)	ดีเพียง 2 ขนาด (3-5)	พังบนมือถือ (0-2)
เมนู/รูปภาพ/ตารางปรับตามอุปกรณ์ (4)	ปรับครบ (4)	ปรับบางส่วน (2-3)	ไม่ปรับ (0-1)
Viewport และ Media Query (2)	ใช้ถูกต้อง (2)	ใช้บางส่วน (1)	ไม่มี (0)

5.6 การเชื่อมต่อ API (12 คะแนน)

เกณฑ์ (คะแนน)	ดีมาก	พอใช้	ต้องปรับปรุง
ดึงข้อมูลและแสดงผลถูกต้อง (6)	เรียกสำเร็จ แสดงผลสวยงาม (6)	แสดงได้แต่ไม่สมบูรณ์ (3-5)	เรียกไม่สำเร็จ (0-2)
Loading State และ Error Handling (4)	มีครบทั้งสอง (4)	มีอย่างใดอย่างหนึ่ง (2)	ไม่มี (0)
ใช้ async/await หรือ Promise (2)	ถูกต้อง (2)	พอใช้ (1)	ผิด (0)

5.7 SEO เบื้องต้น (8 คะแนน)

เกณฑ์ (คะแนน)	ดีมาก	พอใช้	ต้องปรับปรุง
Title และ Meta Description ไม่ซ้ำกัน (3)	ครบทุกหน้า (3)	ครบบางหน้า (1-2)	ไม่มี (0)
Heading และ Alt Text (3)	ถูกต้องทั้งเว็บ (3)	ถูกบางส่วน (1-2)	ไม่ถูกต้อง (0)
Open Graph / ชื่อไฟล์สื่อความหมาย (2)	ครบ (2)	บางส่วน (1)	ไม่มี (0)

5.8 การทดสอบและแก้ไขข้อผิดพลาด (8 คะแนน)

เกณฑ์ (คะแนน)	ดีมาก	พอใช้	ต้องปรับปรุง
ตารางบันทึกการทดสอบ ≥3 รายการ (3)	ครบ ระบุปัญหา-วิธีแก้ชัดเจน (3)	2 รายการ (1-2)	≤1 รายการ/ไม่ส่ง (0)
ผล Lighthouse (หน้าแรก) / W3C (3)	ทุกหมวด ≥80 และแก้ error แล้ว (3)	60-79 (1-2)	<60 หรือไม่ส่ง (0)
ไม่มี Error ใน Console (2)	ไม่มี (2)	มี warning (1)	มี error (0)

5.9 การใช้ AI อย่างรับผิดชอบและ Viva (12 คะแนน)

เกณฑ์ (คะแนน)	ดีมาก	พอใช้	ต้องปรับปรุง
บันทึกการใช้ AI (4)	≥3 รายการ มี Prompt ผลลัพธ์ และสิ่งที่แก้เอง (4)	ครบแต่ไม่ละเอียด (2-3)	ไม่ครบ/ไม่ส่ง (0-1)
อธิบายโค้ดได้ — Viva (8)	อธิบายถูกต้อง เข้าใจหลักการ ตอบคำถามต่อยอดได้ (7-8)	อธิบายได้บางส่วน (4-6)	อธิบายไม่ได้ (0-3)

ส่วนที่ 6: สรุปคะแนนและเกณฑ์ตัดเกรด

หมวด	คะแนนเต็ม
5.1 โครงสร้างเว็บไซต์และ HTML	10
5.2 CSS และการออกแบบ UI	15
5.3 JavaScript และการโต้ตอบ	15
5.4 เฟรมเวิร์คและไลบรารี	8
5.5 Responsive Design	12
5.6 การเชื่อมต่อ API	12
5.7 SEO เบื้องต้น	8
5.8 การทดสอบและแก้ไขข้อผิดพลาด	8
5.9 การใช้ AI และ Viva	12
รวม	**100**

ช่วงคะแนน	ระดับ
80-100	ดีเยี่ยม
70-79	ดี
60-69	พอใช้
50-59	ผ่านเกณฑ์ขั้นต่ำ
ต่ำกว่า 50	ไม่ผ่าน

หมายเหตุ: หากนักศึกษาไม่เข้ารับการสอบ Viva หรือไม่สามารถอธิบายโค้ดหลักได้เลย ผู้สอนอาจพิจารณาปรับลดคะแนนหมวด 5.2-5.6 ตามสัดส่วนส่วนที่อธิบายไม่ได้

ส่วนที่ 7: แบบฟอร์มตรวจให้คะแนนรายบุคคล

ชื่อ-สกุล ..... รหัสนักศึกษา ..... หัวข้อเว็บไซต์ .....

หมวด	เต็ม	ได้	หมายเหตุ/ข้อสังเกต
5.1 โครงสร้างและ HTML	10		
5.2 CSS และ UI	15		
5.3 JavaScript	15		
5.4 เฟรมเวิร์ค	8		
5.5 Responsive	12		
5.6 API	12		
5.7 SEO	8		
5.8 ทดสอบ/แก้ไข	8		
5.9 AI + Viva	12		
รวม	**100**		

เช็กลิสต์ก่อนตรวจ

- ☐ ส่งครบ: โค้ด, README.md, AI_LOG.md, ภาพ Lighthouse
- ☐ เปิดเว็บแล้ว Console ไม่มี error
- ☐ ทดสอบ Device Toolbar 3 ขนาด (375 / 768 / 1280 px)
- ☐ ทดสอบ API ปกติ และจำลองออฟไลน์ (DevTools ⟶ Network ⟶ Offline) เพื่อดู error handling
- ☐ ทดสอบฟอร์มด้วยข้อมูลว่าง/อีเมลผิดรูปแบบ

ลงชื่อผู้ตรวจ ..... วันที่ ..... คะแนนรวม ..... / 100

ส่วนที่ 8: แนวคำถามสอบปากเปล่า (Viva) สำหรับผู้สอน

วิธีใช้: สุ่มชี้โค้ดในไฟล์ของนักศึกษา เลือก 3-4 ข้อจากตารางด้านล่าง ถามต่อยอดว่า "ถ้าเปลี่ยน...จะเกิดอะไรขึ้น"

เพื่อแยกผู้ที่เข้าใจจริงออกจากผู้ที่คัดลอกมา

หัวข้อ	ตัวอย่างคำถาม
HTML/Semantic	ทำไมใช้ <section> แทน ตรงนี้? <main> ต่างจาก <article> อย่างไร?
CSS Layout	ชี้ส่วนที่ใช้ Flexbox/Grid แล้วอธิบายว่า justify-content หรือ grid-template-columns ทำหน้าที่อะไร
Responsive	Media Query บรรทัดนี้ทำงานเมื่อไร? ถ้าลบ <meta viewport> จะเกิดอะไรบนมือถือ?
JavaScript	ฟังก์ชันนี้ถูกเรียกเมื่อไร? addEventListener ผูกกับ element ใด? ถ้าเปลี่ยน querySelector เป็น getElementById ต้องแก้อะไร?
Form Validation	ถ้าผู้ใช้กรอกอีเมลผิดรูปแบบ โค้ดส่วนไหนตรวจจับ และแสดงข้อความอย่างไร?
API	อธิบาย async/await ในโค้ดนี้ ถ้า API ตอบช้า/ล่ม โค้ดส่วนไหนจัดการ? response.ok ใช้ทำอะไร?
SEO	ทำไมต้องมี h1 เดียว? meta description มีผลอย่างไรต่อผลการค้นหา?
การใช้ AI	ให้เล่าตัวอย่างที่ AI ให้โค้ดผิดหรือไม่เหมาะ และนักศึกษาแก้ไขอย่างไร

ส่วนที่ 9: หมายเหตุสำหรับผู้สอน

- แจกรายชื่อหัวข้อและ API ที่แนะนำล่วงหน้าก่อนวันสอบ (ไม่เฉลยโค้ด) เพื่อลดเวลาคิดหัวข้อ
- กำหนดให้แต่ละคนเลือกหัวข้อต่างกัน เพื่อลดความเสี่ยงงานซ้ำกัน
- ตัวอย่าง API ไม่ต้องสมัครคีย์: Open-Meteo (สภาพอากาศ), JSONPlaceholder (ข้อมูลทดสอบ), PokeAPI, Open Library
- เตือนเวลาที่ Checkpoint ตามส่วนที่ 2 (ชั่วโมงที่ 1:30, 2:05 และ 2:45)
- ตรวจสอบก่อนสอบว่าห้องปฏิบัติการเข้าถึงอินเทอร์เน็ต และเว็บของ API / CDN ที่ใช้ไม่ถูกบล็อก
- ควรจัดสอบ Viva ในวันเดียวกันหรือภายใน 1-2 วัน เพื่อให้ผู้สอบจำโค้ดของตนเองได้